Housemate Model Service

Design Document

Date:	29 / Sep / 2026
Author:	Mauricio Gidi
Reviewer(s):	Zachary Tunison

Introduction

This document defines the design of the Housemate Model Service, the component responsible for modeling houses, rooms, occupants, sensors, appliances, and their state. It summarizes the requirements and use cases, defines the public service interface and domain model, and describes implementation details, exception handling, testing, and design risks.

Overview

The Housemate Model Service provides the domain model and state-management API for the Housemate System. It manages multiple houses and their rooms, occupants, sensors, and appliances. Clients use the Service interface to configure these entities, update and query device state, inspect configuration, and calculate current energy use.

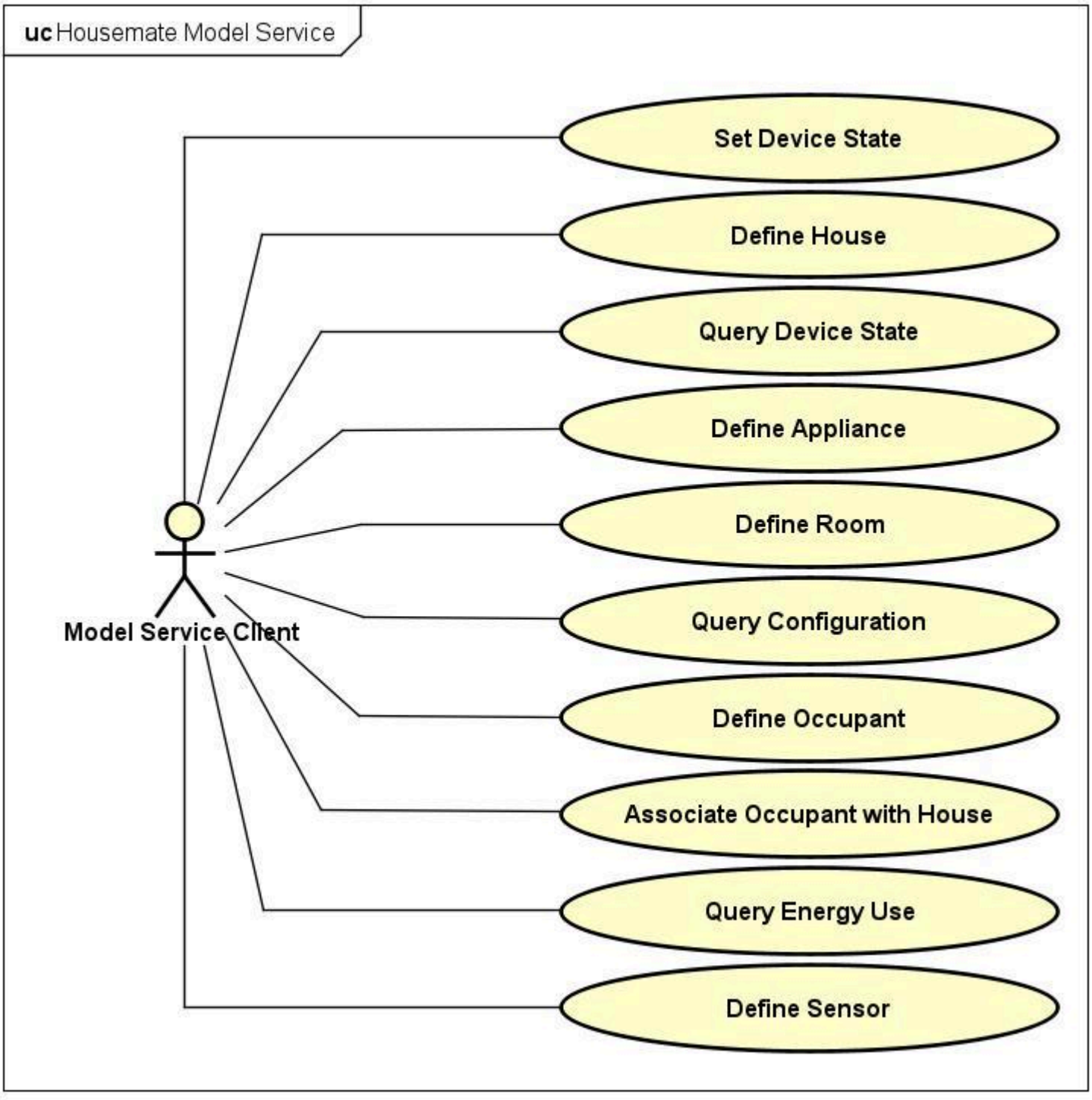
The Model Service does not implement voice processing, autonomous appliance control, or authorization. Those responsibilities belong to other Housemate services. The HousemateModelService integrates with the Assignment 1 KnowledgeGraph to maintain device state while exposing its implementation only through the public Service interface.

Requirements

ID	Requirement	Type	Design Coverage
R1	Model multiple houses, rooms, occupants, sensors, and appliances, including occupants associated with multiple houses.	**F**	Domain classes and associations model the required entities and relationships.
R2	Configure houses, rooms, occupants, sensors, and appliances through the Service API and CLI.	**F**	Service defines configuration operations; CommandLine parses commands and invokes that interface.
R3	Update and query sensor and appliance state.	**F**	setDeviceStatus and showDeviceStatus provide state access through the Service API.
R4	Query configuration for the system, a house, a room, or a device.	**F**	Overloaded showConfiguration operations provide configuration queries at each required level.
R5	Calculate current energy use for an appliance, room, house, and the complete system.	**F**	Appliance energy use is calculated from configured wattage and power state, then aggregated through the domain hierarchy.
R6	Maintain sensor and appliance state in the Assignment 1 KnowledgeGraph.	**NF**	HousemateModelService uses KnowledgeGraph for device-state storage and queries.
R7	Accept an auth_token on all public Model Service API operations.	**NF**	Every Service operation includes an authorization-token parameter for future access control.
R8	Expose a Java service interface and encapsulate the implementation behind a Singleton service instance.	**NF**	HousemateModelService implements Service; clients depend on Service and obtain the implementation through getInstance().

Use Cases

This design supports the following use cases:



The Model Service Client represents any client using the public Service interface, including the command-line interface. The supported use cases cover entity configuration, occupant-to-house association, device-state update and query, configuration query, and energy-use query (R1–R5).

Implementation

The Service interface defines the public API for the Housemate Model Service. HousemateModelService implements this interface and provides the static getInstance() factory method for obtaining the Singleton service instance. External clients depend on Service rather than directly on HousemateModelService, which encapsulates the implementation as required by the system architecture (R8).

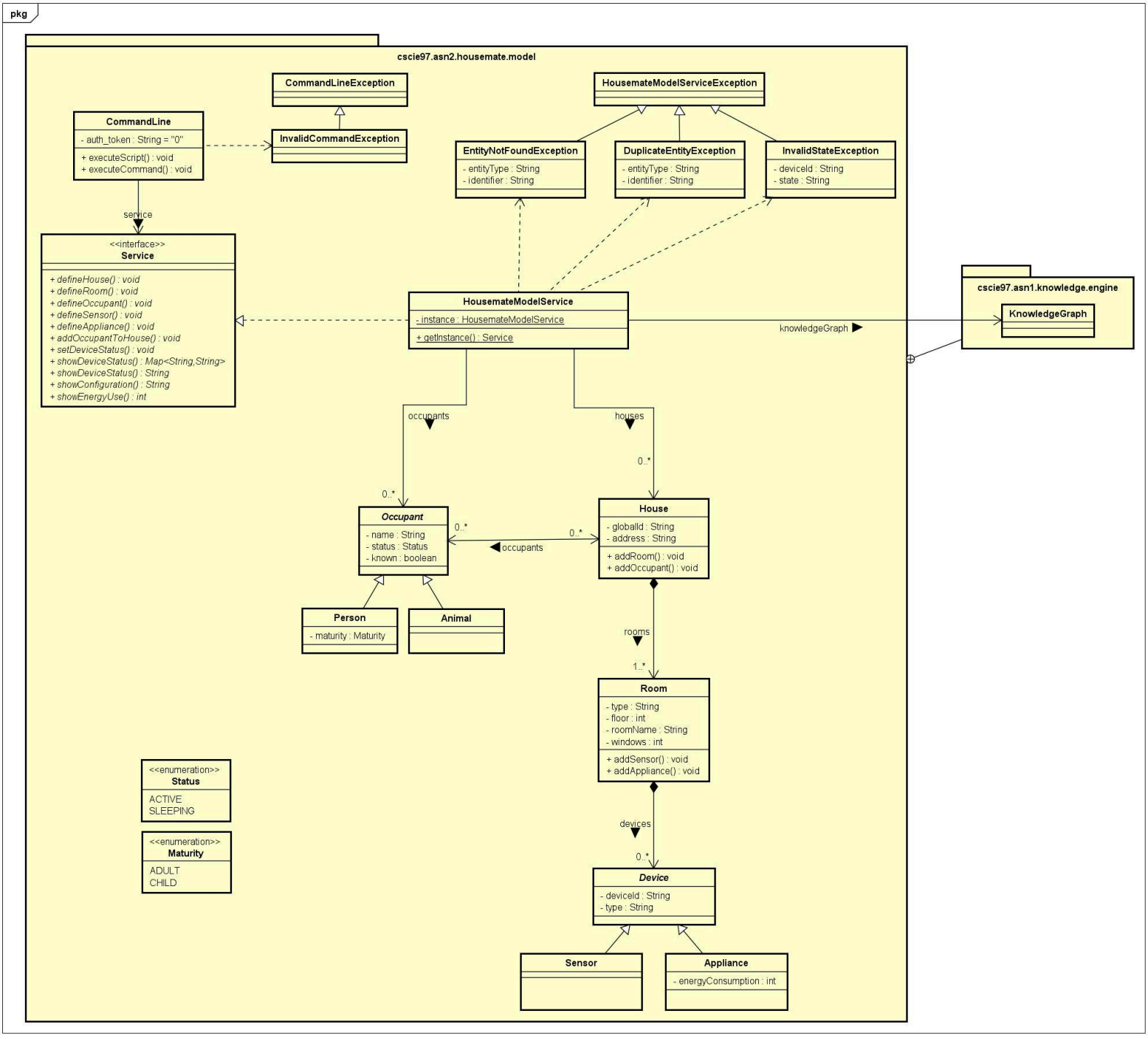
CommandLine provides the command-line interface. It parses and validates commands from a configuration script and translates valid commands into calls on Service. CommandLine therefore depends only on the public service interface and does not directly manipulate domain objects.

HousemateModelService manages the top-level collections of houses and occupants. A House contains Rooms and maintains associations to its Occupants. A Room contains Devices, with Sensor and Appliance specializing the abstract Device class. Person and Animal similarly specialize Occupant.

Structural configuration is represented by the domain objects and is also recorded in the KnowledgeGraph where required. Dynamic sensor and appliance status is stored in the KnowledgeGraph and accessed through the Model Service API (R3, R6). Every public Service operation accepts an authToken parameter; Assignment 2 carries the token through the API but does not yet perform authorization (R7).

Class Diagram

The following class diagram defines the classes defined in this design.



The class diagram defines the public Service interface, its HousemateModelService implementation, the CommandLine client, the domain model, and the Model Service exception hierarchy. House composes one or more Rooms, and each Room composes zero or more Devices. Occupants may be associated with multiple Houses. Sensor and Appliance specialize Device, while Person and Animal specialize Occupant. HousemateModelService integrates the domain model with the external Assignment 1 KnowledgeGraph.

Class Dictionary

This section specifies the class dictionary for each of the classes defined in the class diagram.

Service

Purpose: Public API for configuring and querying the Model Service.

Methods: All operations throw HousemateModelServiceException.

- defineHouse(String authToken, String houseName, String address): void — Defines a house.
- defineRoom(String authToken, String roomName, int floor, String roomType, String houseName, int windows): void — Defines a room.
- defineOccupant(String authToken, String occupantName, String occupantType): void — Defines an occupant.
- defineSensor(String authToken, String sensorName, String sensorType, String houseName, String roomName): void — Defines a sensor.
- defineAppliance(String authToken, String applianceName, String applianceType, String houseName, String roomName, int energyUse): void — Defines an appliance.
- addOccupantToHouse(String authToken, String occupantName, String houseName): void — Associates an occupant with a house.
- setDeviceStatus(String authToken, String deviceType, String houseName, String roomName, String deviceName, String statusName, String value): void — Sets one device state.
- showDeviceStatus(String authToken, String deviceType, String houseName, String roomName, String deviceName): Map<String,String> — Returns all device states.
- showDeviceStatus(String authToken, String deviceType, String houseName, String roomName, String deviceName, String statusName): String — Returns one device state.
- showConfiguration(...) : String — Returns system, house, room, or device configuration through four overloads.
- showEnergyUse(...) : int — Returns system, house, room, or appliance energy use through four overloads.

Associations: Implemented by HousemateModelService; used by CommandLine.

HousemateModelService

Purpose: Singleton implementation of Service and coordinator of the domain model.

Properties: instance: HousemateModelService {static}; knowledgeGraph: KnowledgeGraph; houses: Map<String,House>; occupants: Map<String,Occupant>.

Methods: getInstance(): Service; implements all Service operations; private lookup and device-type validation helpers.

Associations: Realizes Service; manages 0..* Houses and 0..* Occupants; depends on external Assignment 1 KnowledgeGraph and Triple.

CommandLine

Purpose: Parses and executes Housemate configuration commands through Service.

Properties: service: Service; auth_token: String = "0".

Methods: CommandLine(Service); executeScript(String): void; executeCommand(String): void.

Associations: Holds one Service reference.

House

Purpose: Models one house with its rooms and occupants.

Properties: globalId: String; address: String; rooms: Map<String,Room>; occupants: Map<String,Occupant>.

Methods: addRoom(Room); getRoom(String); getGlobalId(); addOccupant(Occupant); getAddress(); getRooms(); getOccupants().

Associations: Composes 1..* Rooms; associates with 0..* Occupants.

Room

Purpose: Models one room and its devices.

Properties: type: String; floor: int; roomName: String; windows: int; devices: Map<String,Device>.

Methods: addSensor(String,String); addAppliance(String,String,int); getDevice(String); getName(); getType(); getFloor(); getWindows(); getDevices().

Associations: Owned by one House; composes 0..* Devices.

Occupant

Purpose: Abstract base class for people and pets.

Properties: name: String; status: Status = ACTIVE; known: boolean = true.

Methods: getName(): String; getType(): String {abstract}.

Associations: May be associated with 0..* Houses; generalized by Person and Animal. Status and known are modeled but Assignment 2 exposes no public commands to modify them.

Person

Purpose: Models an adult or child.

Properties: maturity: Maturity.

Methods: Person(String,Maturity); getType(): String.

Associations: Extends Occupant.

Animal

Purpose: Models a pet.

Properties: Uses inherited Occupant properties.

Methods: Animal(String); getType(): String returns PET.

Associations: Extends Occupant.

Device

Purpose: Abstract base class for sensors and appliances.

Properties: deviceId: String; type: String.

Methods: getId(): String; getType(): String.

Associations: Owned by one Room; generalized by Sensor and Appliance.

Sensor

Purpose: Models a sensor.

Properties: Uses inherited Device properties.

Methods: Sensor(String deviceId, String type).

Associations: Extends Device.

Appliance

Purpose: Models an appliance and its configured energy use.

Properties: energyConsumption: int, plus inherited Device properties.

Methods: Appliance(String deviceId, String type, int energyConsumption); getEnergyConsumption(): int.

Associations: Extends Device.

Status

Purpose: Occupant status enumeration.
Values: ACTIVE, SLEEPING.

Maturity

Purpose: Person maturity enumeration.
Values: ADULT, CHILD.

HousemateModelServiceException

Purpose: Base checked exception for Model Service errors.
Methods: Constructors support empty, message, cause, or message + cause forms.
Associations: Parent of EntityNotFoundException, DuplicateEntityException, and InvalidStateException.

EntityNotFoundException

Purpose: Reports a required house, room, occupant, device, or expected device type that cannot be found.
Properties: entityType: String; identifier: String.
Methods: EntityNotFoundException(String,String); getEntityType(); getIdentifier().
Associations: Extends HousemateModelServiceException.

DuplicateEntityException

Purpose: Reports an attempt to define an entity that already exists.
Properties: entityType: String; identifier: String.
Methods: DuplicateEntityException(String,String); getEntityType(); getIdentifier().
Associations: Extends HousemateModelServiceException.

InvalidStateException

Purpose: Reports a requested device state that has not been defined.
Properties: deviceId: String; state: String.
Methods: InvalidStateException(String,String); getDeviceId(); getState().
Associations: Extends HousemateModelServiceException.

CommandLineException

Purpose: Base checked exception for CLI and script-processing errors.
Methods: Constructors support empty, message, cause, or message + cause forms.
Associations: Parent of InvalidCommandException.

InvalidCommandException

Purpose: Reports invalid or unsupported command syntax.
Methods: Constructors support empty, message, cause, or message + cause forms.
Associations: Extends CommandLineException.

> **External dependency note:** KnowledgeGraph and Triple are Assignment 1 classes. They are shown as dependencies but are not part of this class dictionary.

Implementation Details

HousemateModelService maintains registries of Houses and Occupants. Houses contain Rooms, and Rooms contain Sensors and Appliances. Identifiers are hierarchical: house identifiers are global, room names are unique within a house, and devices are addressed externally as house:room:device. This structure supports the required configuration and lookup operations (R1, R2, R4).

The KnowledgeGraph stores device-state values using the device's hierarchical identifier. State predicates use the form status:<statusName>; for example, the power state of house1:kitchen:oven1 is stored under status:power. This allows different sensor and appliance types to expose different state names without adding a separate Java field for every possible device state (R3, R6).

Each Appliance stores its configured energy consumption in Watts. Current appliance energy use is its configured value when status:power is ON and zero otherwise. Room energy use sums its active appliances, House energy use sums its rooms, and system energy use sums all houses (R5).

The CLI validates the documented command forms and invokes the corresponding Service operations. The supplied housesetup.txt contains appliance definitions both with and without energy-use values, so the CLI accepts either form. When energy-use is omitted, the value defaults to 0 W.

Every Service operation accepts authToken as required by the architecture. Assignment 2 does not interpret or validate the token; authorization can be added later without changing the public API (R7).

Exception Handling

The design separates command-line errors from Model Service errors. CommandLineException is the base checked exception for CLI and script-processing failures, and InvalidCommandException identifies invalid or unsupported command syntax.

HousemateModelServiceException is the base checked exception for service-level failures. EntityNotFoundException identifies a missing house, room, occupant, or device; DuplicateEntityException identifies an attempt to create an existing entity; and InvalidStateException identifies a requested device state that has not been defined.

EntityNotFoundException and DuplicateEntityException record the entity type and identifier involved in the error. InvalidStateException records the device identifier and requested state.

When a script is executed, CommandLine reports InvalidCommandException and HousemateModelServiceException errors with the input line number and continues processing subsequent commands. A failure to open or read the script itself raises CommandLineException and stops script processing.

Testing

Testing uses TestDriver and command scripts to verify the component.

- Functional: Verify houses, rooms, occupants, sensors, appliances, occupant associations, device state, configuration queries, and energy-use queries.
- Energy use: Verify ON appliances contribute their configured Watts; OFF or unset appliances contribute 0 W; verify room, house, and system totals.
- Exception handling: Test duplicates, missing entities, wrong device types, missing states, malformed commands, and unreadable script files. Line-level errors must not stop later commands.
- Regression: Recompile and rerun valid and error scripts after changes.
- Performance: Configure multiple houses, rooms, and devices and confirm queries remain responsive. No explicit performance target is specified.
- Packaging: Unzip the final submission into a clean directory, then compile and run with the required commands.

Required commands:

```
javac cscie97/asn2/housemate/model/*.java cscie97/asn2/housemate/test/*.java
java -cp . cscie97.asn2.housemate.test.TestDriver housesetup.txt
```

Risks

The main implementation risks are:

- Domain and KnowledgeGraph state could become inconsistent. HousemateModelService is the single update path; define, set, and show operations verify both representations.
- Hierarchical identifiers or state names could be built inconsistently. The design uses house:room:device IDs and status:<name> predicates throughout.
- Missing appliance energy-use values could undercount usage. The CLI accepts the supplied file format and defaults omitted values to 0 W.
- Energy use depends on status:power. ON returns the configured wattage; OFF or missing power state returns 0 W.

These risks are validated with configuration, state, exception, and energy-use tests.